Project 1 – Lexical Analysis

Ayden Herring

Parker Riggs

Regular Expressions and Tokens

We tried to tokenize as much as possible with major focus on most seen elements observed in the student example files. Because these were intro level student projects, patterns were easily recognizable to tokenize pretty much all relevant syntax. We found this by reviewing almost half of the projects to determine the most common implementations and some outliers that may help to prove plagiarism.

First regex is just to remove whitespace and comments for better comparison (these are not tokenized and do not return anything)

```
[ \t\r\n]+ { }
"//".* { }
"/*"([^*]|\*+[^*/])*"*/" { }
```

We tokenize include files to see if some weird include file is added that maybe only a select few others use that could hint at plagiarism. Basically, capture anything that starts with a #include. Shouldn't matter that specific ones aren't captured as further review can be done if several includes are matched to another file with several includes.

```
#"include"[^\n]* { printf("001 "); }
```

We tokenize basic control keywords like if, else, for, while, etc. Just the most basic / common as these are intro projects. And “do” as the outlier as found in review of projects.

```
"if" { printf("010 "); }
"else" { printf("011 "); }
"for" { printf("012 "); }
"do" { printf("013 "); }
"while" { printf("014 "); }
"return" { printf("015 "); }
"break" { printf("016 "); }
"continue" { printf("017 "); }
```

We tokenize basic types (chosen by the type of project) this project doesn't require any more types than int void and char.

```
"int" { printf("020 "); }
"void" { printf("021 "); }
```

```
"char" { printf("022 "); }
```

We tokenize literals (strings) that are “hard coded” like in a cout statement as the format of those could be copied and structurally the same but reworded differently.

```
\"([^\"\\]|\\.)*\" { printf("030 "); }
'([^'\\]|\\.)' { printf("031 "); }
```

We tokenize identifiers like variable names and function names as well as these could also be structurally the same but reworded differently.

```
[a-zA-Z_][a-zA-Z0-9_]* { printf("040 "); }
```

We tokenize numbers as they appear in every project but could be found arranged in a similar way in another project.

```
[0-9]+"."[0-9]+ { printf("050 "); }
[0-9]+ { printf("050 "); }
```

We tokenize operators as things could be written differently but fundamentally do the same thing like + is basically ++

```
"++"|"--"|"+"|"-"|"*"|"/"|"%"|"=="|"!="|"="|"<="|">="|"<"|">"|"&&"|"||"|"!"|"&" { printf("060 "); }
```

We tokenize delimiters to catch how student may have hid how they for example restructured the format of a function call but showing there it starts and ends or if another student blatantly copied another

```
[()\{\}\[\];,:] { printf("070 "); }
```

We then tokenize any unrecognized character

```
. { printf("999 "); }
```

Winnowing Algorithm

After lexical analysis, each source file is represented as a sequence of numeric tokens. The detector then applies a windowing based fingerprinting process that compares submissions at a deeper, structural level rather than just exact text matching.

First, all tokens for a file are concatenated into one token stream. Using this stream, overlapping k grams are generated and hashed together into integer values. In our implementation, k is fixed to 12 characters, this preserves the structure while reducing the sensitivity to things like renamed variables, spacing changes, and other small edits that could be made to throw off a traditional algorithm.

Next, a sliding window is moved across the sequence of hashes. For each window, the minimum hash value is selected as a fingerprint. We use a window size of 5. This step reduces the number of hashes while still preserving the patterns from across the file. Fingerprints are stored in a set, so duplicates are automatically removed.

To compare two submissions, we compute the following:

1. Intersection: # of shared fingerprints
2. Union: Total unique fingerprints across both submissions
3. Similarity Score: Shared divided by union

This effectively is a jaccard style similarity measure over fingerprint sets. Scores range from 0 to 1, values closer to 1 indicate a stronger similarity.

Experimental Setup

We tested the detector on the provided example student set. This was the workflow:

1. Run lexical scanner to produce token streams for each submission.
2. Build fingerprints for each submission with k = 12 and window size set = 5
3. Compute pairwise similarity scores using shared and total fingerprints.
4. Sort and inspect the highest ranked pairs as mostly likely plagiarized candidates.

This setup was chosen mostly because it is fully reproducible and uses the same preprocessing for every file.

Results

The detector produced a ranked list of file pairs with similarity scores and supporting counts. Generally, the highest ranked pairs shared the largest portions of fingerprint structure, while unrelated programs had way lower overlap.

With how our Makefile was set up, we were just able to run make run and get the topmost suspicious pairs directly from the report. Here are the top ones:

1. bills_03.c vs bills_09.c | similarity=1.0000 | shared=98 | total=98

2. bills_03.c vs bills_53.c | similarity=1.0000 | shared=98 | total=98

3. bills_03.c vs bills_54.c | similarity=1.0000 | shared=98 | total=98

4. bills_09.c vs bills_53.c | similarity=1.0000 | shared=98 | total=98
5. bills_09.c vs bills_54.c | similarity=1.0000 | shared=98 | total=98
6. bills_53.c vs bills_54.c | similarity=1.0000 | shared=98 | total=98
7. bills_21.c vs bills_38.c | similarity=0.9612 | shared=99 | total=103
8. bills_10.c vs bills_30.c | similarity=0.9569 | shared=111 | total=116
9. bills_35.c vs bills_39.c | similarity=0.9508 | shared=116 | total=122
10. bills_26.c vs bills_37.c | similarity=0.9495 | shared=94 | total=99

These results show that token level windowing can identify likely copied structures even when things like the variable names and formatting are different.

Conclusion

This project demonstrates that lexical tokenization in combination with the winnowing process is a strong and effective baseline for detecting plagiarism especially in student's code. The system scales many different pairwise comparisons, highlighting likely matches and reduces manual review effort by ranking the most suspicious pairs first. While not a standalone verdict, it does provide strong evidence for further investigation purposes and can be extended with even more analysis for even higher accuracy.